

Protocolo de Trabajo con Git, GitHub y Linear

Taller de Integración II y Taller de Integración IV

Segundo semestre 2026

1. Objetivo

Esta guía fija la forma en que los equipos de Taller de Integración II (TI2) y Taller de Integración IV (TI4) trabajarán en el proyecto.

El flujo busca dos cosas concretas: que el trabajo se pueda seguir desde Linear hasta GitHub y que cada equipo conserve sus responsabilidades técnicas.

El proyecto usa tres herramientas:

- **Linear**: planificación, asignación y seguimiento del trabajo.
- **Git**: control de versiones.
- **GitHub**: alojamiento del código, Pull Requests, revisión de código e integración continua.

La división de responsabilidades es directa:

Linear gestiona el trabajo y GitHub gestiona el código.

2. Organización de los equipos

El proyecto estará compuesto por dos equipos.

2.1. Taller de Integración II

TI2 tendrá seis estudiantes y se encargará de:

- desarrollo de la aplicación web;
- desarrollo del backend y API mediante arquitectura por capas;
- implementación de los casos de uso y reglas de negocio compartidas;
- diseño e implementación de la base de datos;
- pruebas asociadas a sus componentes;
- despliegue de sus propios servicios;
- documentación técnica correspondiente.

2.2. Taller de Integración IV

TI4 tendrá tres estudiantes y se encargará de:

- desarrollo de la aplicación móvil;
- planificación general del proyecto;
- preparación y priorización inicial del trabajo;
- coordinación entre ambos equipos;
- control de calidad;
- revisión de código;

- seguimiento de dependencias;
- apoyo técnico e inducción al equipo TI2 cuando sea necesario.

TI4 debe ayudar a TI2 a resolver sus tareas cada vez con más autonomía. En una revisión, TI4 señala el problema y explica el criterio. La persona autora hace el cambio en su propia rama.

3. Repositorio GitHub

Todo el producto vivirá en **un único repositorio GitHub**. El monorepo contendrá las aplicaciones, el backend y los paquetes compartidos.

```
Proyecto
|
+-- apps/
| +-- web/ Aplicación web de TI2
| +-- mobile/ Aplicación móvil de TI4
|
+-- convex/ Backend por capas y base de datos compartidos
| +-- presentation/ Funciones públicas de Convex
| +-- application/ Casos de uso
| +-- domain/ Reglas de negocio
| +-- infrastructure/ Persistencia y servicios externos
+-- packages/ Código compartido cuando corresponda
+-- .github/workflows/ Integración continua
+-- package.json
+-- bun.lock
```

Cada equipo desarrolla, prueba y despliega los componentes que tiene asignados dentro del monorepo.

Un repositorio común no elimina la separación de responsabilidades. TI2 mantiene la aplicación web y el backend. TI4 mantiene la aplicación móvil.

La integración entre los equipos pasa por la API pública de la capa de Presentación del backend. Ambas aplicaciones consumen sus tipos generados. No se copian implementaciones ni se mantienen contratos externos.

3.1. Arquitectura por capas

Las dependencias deben avanzar en esta dirección:

```
Presentación
  |
  v
Aplicación
  |
  v
Dominio
  ^
  |
Infraestructura
```

En el backend, las funciones públicas de Convex reciben la solicitud y llaman a un caso de uso. Los casos de uso coordinan la operación. El Dominio contiene las reglas. Infraestructura conecta con la persistencia, la autenticación y los servicios externos.

Una branch o Pull Request que cruce varias capas debe explicar qué cambia en cada una. React, Mobile y las funciones públicas no deben resolver necesidades de negocio con consultas directas a la base de datos ni con funciones extensas.

4. Uso de Linear

Todo el proyecto usará **un único workspace de Linear**.

Dentro de este workspace existirán dos Teams:

- **TI2:** Web, Backend y Base de Datos.
- **TI4:** Gestión y Aplicación Móvil.

Los dos Teams forman parte del mismo proyecto.

4.1. Triage

TI4 hará el *triage* inicial. Para cada issue debe:

- crear o revisar los issues necesarios;
- asignarlos al Team correspondiente;
- definir su prioridad;
- verificar que tengan una descripción suficiente;
- establecer criterios de aceptación;
- identificar dependencias entre ambos equipos;
- asignarlos al Cycle correspondiente.

Cuando un issue esté listo en el backlog, el equipo responsable organiza su ejecución.

4.2. Linear como fuente de verdad

Toda tarea técnica que requiera trabajo significativo debe tener un issue en Linear.

WhatsApp, Discord y otros canales sirven para conversar, no para asignar trabajo de manera formal. Si una conversación genera una tarea, alguien debe registrarla en Linear.

4.3. Sub-issues

El proyecto no usa **sub-issues** de Linear.

Para una tarea con varios pasos pequeños, usa una lista en la descripción del issue.

Por ejemplo:

TI2-37 Implementar autenticación

- Crear endpoint de login
- Validar credenciales
- Generar token
- Agregar pruebas
- Documentar endpoint

Si una parte del trabajo necesita asignación, seguimiento o revisión propios, conviértela en un issue independiente.

Así se evita gastar issues del plan en tareas que no necesitan seguimiento separado.

5. Cycles y Sprints

Cada *Cycle* de Linear representa un Sprint. No crearemos un Cycle por semana.

La correspondencia es esta:

Cycle 1 -> Sprint 1
Cycle 2 -> Sprint 2
Cycle 3 -> Sprint 3
Cycle 4 -> Sprint Final

Las semanas de cada Sprint quedan para seguimiento, reuniones y control de avance. Al cerrar un Cycle, el equipo revisa:

- trabajo completado;
- trabajo pendiente;
- issues bloqueados;
- deuda técnica generada;
- dependencias entre TI2 y TI4;
- trabajo que debe trasladarse al siguiente Sprint.

6. Estados de los Issues

El workflow tendrá pocos estados:

Backlog → Todo → In Progress → In Review → Done

También puedes usar:

Canceled

Los estados significan lo siguiente:

Backlog: trabajo identificado pero todavía no comprometido.

Todo: trabajo seleccionado para el Cycle actual.

In Progress: existe trabajo activo sobre el issue.

In Review: existe uno o más Pull Requests pendientes de revisión.

Done: todo el trabajo requerido por el issue fue integrado.

Canceled: el trabajo dejó de ser necesario.

Cuando sea posible, la integración Linear-GitHub cambiará estos estados de forma automática.

7. Integración de Linear con GitHub

El repositorio del monorepo debe conectarse al workspace de Linear.

Cada issue conserva el Team responsable, TI2 o TI4. El identificador distingue el área aunque todo el código viva en el mismo repositorio.

Los identificadores de Linear relacionan el código con el trabajo.

Por ejemplo:

TI2-37 TI4-18

Un Pull Request puede referenciar el issue desde la branch, el título o la descripción. La trazabilidad queda así:

Issue Linear → Branch → Commits → Pull Request → Review → Merge

8. Estrategia de ramas

El repositorio usará **feature branches pequeñas y de corta duración**.

La única rama permanente es:

```
main
```

No habrá una rama permanente **develop**. Tampoco usaremos Git Flow con ramas permanentes de desarrollo, release o hotfix.

8.1. Creación de una rama

Antes de empezar una tarea:

1. confirma que existe un issue en Linear;
2. actualiza la rama **main**;
3. crea una rama desde **main**;
4. trabaja solo en el objetivo del issue.

Flujo conceptual:

```
main
|
+-- rama TI2-37-login-web
|
+-- rama TI2-42-auth
|
+-- rama TI4-18-login-mobile
```

9. Nombres de ramas

Usa, de preferencia, el nombre de branch que genera Linear.

El nombre debe conservar:

- nombre o identificador del desarrollador;
- identificador del issue de Linear;
- una descripción breve.

Ejemplo:

```
vriviera/ti2-37-login
```

Si Linear genera un nombre demasiado largo, puedes abreviarlo.

Por ejemplo, en lugar de:

```
vriviera/ti2-37-implementar-endpoint-de-autenticacion-de-usuarios
```

usa:

```
vriviera/ti2-37-login
```

Conserva siempre el identificador de Linear.

No uses:

```
login
feature1
cambios
prueba
rama-nueva
```

10. Tamaño de las ramas

Una rama debe resolver **una intención concreta**.

Mantén las ramas pequeñas para facilitar:

- revisión;
- pruebas;
- resolución de conflictos;
- integración continua;
- detección de errores;
- reversión de cambios.

No fijaremos un límite artificial de líneas modificadas. El criterio es que otra persona pueda entender y revisar el Pull Request sin encontrarse con varias funcionalidades independientes.

11. Pull Requests

Todo cambio que llegue a `main` debe pasar por un **Pull Request** en el repositorio del monorepo.

Nadie puede hacer push directo a `main`.

El Pull Request debe indicar:

- qué problema resuelve;
- qué cambios introduce;
- cómo puede probarse;
- issue de Linear relacionado;
- cualquier consideración relevante para el reviewer.

Ejemplo:

```
TI2-37: Implementar login de usuarios
```

La descripción debe dar el contexto suficiente para revisar el cambio sin reconstruirlo desde el código.

12. Relación entre Issues y Pull Requests

Un issue y un Pull Request no tienen que mantener una relación 1:1.

Un issue representa una **unidad de trabajo**.

Un Pull Request representa una **unidad de integración de código**.

Un issue puede requerir varios Pull Requests.

Ejemplo:

```
TI2-42 Autenticación
```

```
|
+-- PR 1: modelo de usuario
|
+-- PR 2: endpoint login
|
+-- PR 3: refresh token
```

El issue termina cuando todo el trabajo necesario está integrado.

13. Stacked Pull Requests

Si no puedes dividir una tarea en cambios independientes, encadena Pull Requests o usa *stacked PRs*.

Ejemplo:

```
main
|
+-- PR A
    |
    +-- PR B
        |
        +-- PR C
```

Esta técnica es una excepción. No es el flujo predeterminado.

Como límite práctico:

usa como máximo 3 stacked PRs; el límite absoluto es 4.

Si una cadena supera cuatro Pull Requests dependientes, revisa la planificación y divide la tarea de nuevo.

14. Revisión de código

Otra persona debe revisar cada Pull Request antes de integrarlo.

Una revisión puede terminar en:

- aprobación;
- comentarios;
- solicitud de cambios.

La revisión debe comprobar:

- cumplimiento del issue;
- corrección;
- claridad del código;
- mantenibilidad;
- errores potenciales;
- pruebas;
- seguridad;
- respeto de la dirección de dependencias entre capas;
- ubicación correcta de las reglas de negocio;
- compatibilidad con otros componentes;
- contrato de API cuando corresponda.

15. Progresión de las revisiones de TI2

TI2 está en una etapa inicial de formación, así que la supervisión cambiará durante el semestre.

Etapas inicial

Durante la primera etapa, integrantes de TI4 revisarán los Pull Requests de TI2.

TI4 puede pedir cambios, explicar el problema y sugerir una solución. La persona autora hace los cambios en su propia rama.

Etapa intermedia

En la etapa intermedia, TI2 empezará a revisar código entre pares. TI4 seguirá revisando directamente los cambios relacionados con:

- API;
- arquitectura;
- base de datos;
- seguridad;
- integración con móvil;
- cambios con impacto significativo.

Etapa avanzada

En la etapa avanzada, TI2 debería revisar e integrar sus cambios con autonomía. TI4 se concentrará en:

- coordinación;
- QA;
- arquitectura;
- integración;
- resolución de dependencias.

16. Política de Merge

Usaremos **Squash and Merge**. Así, cada Pull Request deja un solo commit en **main**, aunque la branch tenga commits intermedios.

Después del merge:

- conserva la rama de desarrollo;
- actualiza el issue relacionado;
- incluye la documentación asociada.

Las ramas integradas forman parte del historial de trabajo y de la evidencia de participación de cada integrante. Desactiva en GitHub la eliminación automática de branches después del merge.

17. Protección de main

Protege la rama **main** con las opciones disponibles en GitHub.

Como mínimo, exige:

- no permitir push directo;
- exigir Pull Request;
- exigir al menos una aprobación;
- exigir que las validaciones automáticas requeridas finalicen correctamente;
- exigir resolución de conversaciones pendientes antes del merge.

Estas reglas aplican a TI2 y TI4. El equipo TI4 también debe pasar por Pull Request.

18. Integración Continua

GitHub Actions ejecutará las verificaciones de cada Pull Request.

Las rutas modificadas determinan qué verificaciones se ejecutan. Como mínimo:

- instalar las dependencias desde el `package.json` y el único `bun.lock` de la raíz;
- ejecutar el build de la aplicación o paquete afectado;
- formatter;
- linter;
- pruebas unitarias del Dominio y de los casos de uso;
- pruebas de integración de Infraestructura y de Presentación cuando corresponda.

Si un cambio afecta el backend, Web, Mobile o un paquete compartido, la CI ejecuta las verificaciones de todos los componentes afectados.

No integres un Pull Request con la CI fallando, salvo una excepción justificada.

19. Integración entre TI2 y TI4

TI2 implementará y desplegará los servicios compartidos con **Convex** y arquitectura por capas. Convex ofrece los puntos de entrada, la ejecución y la persistencia que usan las aplicaciones. Los casos de uso y las reglas de negocio no deben acoplarse a Convex. Better Auth funcionará como adaptador de Infraestructura.

El monorepo permite que TI2 y TI4 modifiquen una funcionalidad en el mismo Pull Request cuando necesiten coordinarse. También pueden separar los cambios si la revisión o el despliegue lo justifican. En ambos casos, relacionen los cambios en Linear.

Cuando un cambio de TI2 afecte una funcionalidad de TI4:

- relaciona el cambio con un issue de Linear;
- identifica al equipo afectado;
- comunica los cambios incompatibles antes de integrarlos;
- actualiza la documentación del servicio;
- indica si afecta Presentación, Aplicación, Dominio o Infraestructura;
- actualiza autenticación, tipos o estructura de datos si corresponde.

20. Cambios en el modelo de datos

Convex administra los datos como parte de Infraestructura. Versiona en `convex/` el esquema, las funciones, las relaciones y las estructuras de datos. Mantén las entidades y reglas del negocio en Dominio, no en el esquema ni en la interfaz. Toda configuración necesaria para ejecutar el sistema debe quedar documentada y versionada.

Si un cambio del modelo afecta una funcionalidad de TI4, registra esa dependencia entre equipos en Linear.

21. Dependencias entre equipos

Si una tarea depende del trabajo del otro equipo, registra la dependencia en Linear.

Ejemplo:

TI4-25 Pantalla de recuperación de clave

Bloqueada por:

TI2-61 Endpoint de recuperación de clave

TI4 hará seguimiento de estas dependencias durante la coordinación del proyecto.

22. Definición de Done

Un issue no termina porque “el código funciona en el computador del desarrollador”. Marca una tarea como **Done** cuando:

- cumple sus criterios de aceptación;
- el código fue integrado mediante Pull Request;
- las revisiones fueron resueltas;
- CI finalizó correctamente;
- las pruebas necesarias fueron realizadas;
- la documentación fue actualizada cuando corresponde;
- las dependencias entre capas respetan la arquitectura definida;
- no deja dependencias conocidas sin registrar;
- el cambio está disponible en **main**.

23. Flujo de trabajo completo

El flujo estándar es:

1. TI4 hace el triage inicial del trabajo.
2. Linear asigna el issue al Team y al Cycle correspondientes.
3. El integrante toma la tarea desde Linear.
4. Identifica las capas afectadas y el punto de entrada de la funcionalidad.
5. Actualiza localmente **main**.
6. Crea desde Linear una branch asociada al issue dentro del monorepo.
7. Implementa el cambio.
8. Hace commits en su branch.
9. Publica la branch en GitHub.
10. Abre un Pull Request.
11. Relaciona el Pull Request con el issue en Linear.
12. Ejecuta CI sobre el cambio.
13. Otro integrante revisa el código.
14. Si hay observaciones, el autor modifica su propia branch.
15. Cuando CI y la revisión están aprobadas, hace Squash and Merge.
16. Conserva la branch utilizada en el repositorio.
17. Actualiza el estado del trabajo en Linear.
18. Despliega el cambio cuando corresponda.

El flujo queda así:

```
Linear
|
v
Issue
|
v
Branch pequeña
|
v
Desarrollo
|
v
Pull Request
|
+----> CI
```

```

|
+----> Code Review
|
v
Squash Merge
|
v
main
|
v
Deploy

La branch se conserva.

```

24. Stack tecnológico

Ambos equipos deben conocer el stack tecnológico, aunque cada uno mantenga componentes distintos. El proyecto usa:

Componente	Tecnología seleccionada
Modelo arquitectónico	Arquitectura por capas
Frontend Web	React + Vite + TanStack Router
Servidor / Backend	Convex
Base de Datos	Convex
Autenticación	Better Auth
Aplicación Móvil	Expo + React Native
Monorepo y dependencias	Bun workspaces
Integración Continua	GitHub Actions
Gestión de Proyecto	Linear
Control de Versiones	Git + GitHub

TI2 despliega y mantiene los servicios compartidos, incluidas las capas de Aplicación, Dominio e Infraestructura implementadas con Convex y la integración de Better Auth. TI4 integra y mantiene la Presentación Mobile y su infraestructura cliente dentro del mismo monorepo. Las bibliotecas auxiliares y las configuraciones internas pueden evolucionar, pero cualquier cambio en los contratos o las dependencias entre equipos debe coordinarse antes.

25. Principios generales

Durante el proyecto rigen estos principios:

1. Registra todo trabajo relevante para que pueda seguirse.
2. Nadie hace push directo a **main**.
3. Mantén las ramas pequeñas y de corta duración.
4. Haz que otra persona revise el código.
5. TI4 ayuda a TI2 a ganar autonomía sin reemplazar su trabajo.
6. Usa Linear para representar el estado del trabajo.
7. Usa GitHub para representar el estado del código.
8. Mantén todos los componentes principales del producto en el monorepo.
9. Separa Presentación, Aplicación, Dominio e Infraestructura.
10. Mantén las reglas de negocio fuera de las aplicaciones cliente.
11. Mantén explícitos y versionados la API compartida y los paquetes internos.

12. Haz visibles los problemas cuanto antes.

26. Resumen

El proyecto queda organizado mediante estas relaciones:

Linear = planificación y seguimiento

GitHub = código, revisión, CI e integración

Monorepo = código compartido, trazable y coordinado

El resultado esperado es un proceso que ambos equipos puedan repetir y mantener sin depender de una sola persona.